Félix Arnold

ECE 6100 - Project 3
Cache Coherence

Once our simulation working, we can use it to compare the performance for each available trace. All the outputs have been put together in the table at the last page. We can observe that for all configuration, the fastest protocol in term of cycles count is the MOESIF with

- 4516 cycles for array_block
- 6680 cycles for array_stripe
- 54322 cycles for matmul_row
- 121722 cycles for matmul_col
- 53324 cycles for map_reduce
- 80351 cycles for pipeling_migrating

This result is not surprising as MOESIF disposes of 6 different states for cache blocks which allows it to minimize memory accesses. For each benchmark, we will explain in slightly more details the results we found.

array_block

The execution of array_block first creates an array of random values before using several processors in parallel to compute the sum of the whole array. With this benchmark, each processor only loads the data it needs to sum. Moreover, these data are never modified, only read, and no other processor will need to load them.

MSI and MOSI performs similarly as the Owned state is useful when data are modified and then shared, which is not the case here.

The Exclusive state allows the benchmark to run faster because it decreases the number of memories reads and replace them with cache-to-cache transfers, which are much faster. With MSI and MOSI, the only cache-to-cache transfers happen when writing the variable *partial_sums.* As a matter of fact, all cores access the same cache block whose status is set as Modified, allowing it to have its content transfers from one cache to another. No transfer can happen when a block status is Shared. On the contrary, using the Exclusive state, when a cache line has loaded a cache block from memory and it is the only cache having done so, the line can be set as E, meaning that the cache is able to transfer its content to another cache requesting the same block.

Here, we can notably observe that each cache block is 64 bytes long. As such, we can see in the traces that the values inside *vals* are not aligned with the blocks. Thus, at some point the core 0 will need for example to access a block already loaded and exclusive to the core 1. Thanks to the E state, the core 1 will be able to transfer it to the core 0. Thus, MESI and MOESIF are faster than MSI and MOSI.

The Forwarder state allows even more transfer because once the data in a E cache line has been transferred to another cache, its states become S, making it incapable of transferring the data again. With MOESIF, its state now pass at F, allowing it to continue forwarding its content to other caches while no write happens. Here we can see it happens several times.

array_stripe

The behavior and output of array_stripe has the same goal as array_block. Thus , the reason MOESIF is better is the same.

⇒ We can observe that array_stripe performs much worse than array_block while both algorithms seem to do the exact same thing. This is because in array_block, each core accesses a continuous part of the array *vals,* meaning that the blocks access by a core are accessed by no other cores (if we forget overlaps). On the contrary , with array_stripe, the access to *vals* are alternated between the core. If the core 0 read the value n from the array, the core 1 will read the value $n + 1$. This is highly inefficient because each core need to access all blocks storing the value of *vals,* while, with array_block, each block only accesses around $1/8$ of them. As such, less memory read and/or cache-to-cache transfers, depending on the protocol used, are needed for array_block.

matmul_row

The matmul_row benchmark performs the following matrix calculus: $C = C + A \times B$

The computation is divided between each thread such that each of them only performs the operation for a certain portion of the rows of C.

As previously, the Exclusive and Forwarded states allow to transform memory reads into cache-to-cache transfers as a lot of cache block are accessed by all cores. For instance, all elements of A are access by all cores, and thanks to E and F states, fewer cache needs to load them from memory as the other ones can, while there is still a cache line in E or F state, then transfer the data to the other processors. As previously, the F state allows for more transfers to happen. Moreover, each cache will write several times to the same block and, except overlaps, is the sole cache accessing it. Thus, the E state also allows a cache line to silently upgrade to the M state without needing to broadcast to other nodes, accelerating even more the operations.

The Owned state allows a modified value to be shared with other processors, as long as they only do read operations. Thus, this allows to have fewer write back to and read from memory. It typically happens because of overlaps as the content of C is not aligned with the cache block address. When a cache block containing some part of C has been modified by a processor, it could also be read by another core because this one needs to access another data part of the block.

matmul_col

This benchmark works the same way as matmul_row with the only difference that the operation between the threads are divided between portion of the columns of C.

⇒ We can observe that matmul_col performs worse matmul_row for the same reason as previously with the arrays. The rows of C are continuous part of memory while this is not the case for the columns. Thus, the processor of matmul_col would continuously access and modify cache blocks used by other processors, requiring more cache-to-cache transfers and memory reads (due to memory invalidation), while this phenomenon is much more limited with matmul_row.

mapreduce

We can apply the previous explanation to this trace to explain why MOESIF is better. The access to the queues are random, so that protocol allow for less memory load when several processor access to the same cache block thanks to E and F. The update of *reduction* can also, for example, be optimized as fewer memory read are needed.

pipeline_migrating

Same as previously, MOESIF allow for better optimizations when modifying the *workitems* and removing/inserting them inside the *queues*

Protocol	Trace	Cycles	Cache Misses	Cache Accesses	Silent Upgrades	$-to-$ Transfers	Memory Reads	Memory Writes
MSI	array_block	6340	65	2736	0	7	58	7
MESI	array_block	5493	65	2736	0	17	48	7
MOSI	array_block	6340	65	2736	0	7	58	7
MOESIF	array_block	4516	65	2736	0	29	36	7
MSI	array_stripe	28751	287	2760	0	7	280	7
MESI	array_stripe	26196	287	2760	0	42	245	7
MOSI	array_stripe	28751	287	2760	0	7	280	7
MOESIF	array_stripe	6680	287	2760	0	251	36	7
MSI	mapreduce_pthread	73327	637	11281	0	570	424	570
MESI	mapreduce_pthread	61319	620	11281	0	879	151	462
MOSI	mapreduce_pthread	72421	636	11281	10	579	414	562
MOESIF	mapreduce_pthread	53324	679	11281	12	986	52	512
MSI	matmul_col	227346	3302	118504	0	2744	999	2744
MESI	matmul_col	195788	2800	118504	63	2358	751	2320
MOSI	matmul_col	170598	2565	118504	0	3505	359	1824
MOESIF	matmul_col	121722	1974	118504	65	2062	101	1338
MSI	matmul_row	59906	810	92400	0	553	265	553
MESI	matmul_row	58977	790	92400	65	574	230	540
MOSI	matmul_row	62913	724	92400	0	606	264	327
MOESIF	matmul_row	54322	848	92400	65	759	101	420
MSI	pipeline	108778	836	9438	0	828	567	828
MESI	pipeline	107552	828	9438	7	827	556	813
MOSI	pipeline	84559	818	9483	0	1378	135	726
MOESIF	pipeline	80351	846	9483	7	1343	54	772